

Date _____

Insertion Sort

$O(n^2)$

Insertion(int arr[], int n)

{

int i, j, Key;

for (i=1; i<n; i++)

{

Key = arr[i];

j = i-1;

while (j >= 0 && arr[j] > Key)

{

arr[j+1] = arr[j];

j = j-1;

}

arr[j+1] = Key;

}

}

Bright

Date _____

Merge Sort

$O(n \log n)$

MergeSort(A)

{ $n = \text{len}(A)$

if $n \leq 1$;

return A

$L = \text{MergeSort}(A[0 : \frac{n}{2} - 1])$

$R = \text{MergeSort}(A[\frac{n}{2} : n - 1])$

return MERGE(L, R)

}

MERGE(L, R)

{ result = length n array

$L(n/2) = \infty$; $R(n/2) = \infty$

$i = 0$; $j = 0$

for k in $(0 \dots n-1)$

if $L[i] < R[j]$

result[k] = $L[i]$

$i = i + 1$;

else

result[k] = $R[j]$

$j = j + 1$;

return result;

}

Bright

Date _____

QUICK SORT

$$W = O(n^2)$$

$$A = O(n \log n)$$

QUICKSORT(A, p, r)

{ if { p < r:

 q = PARTITION(A, p, r)

 QUICKSORT(A, p, q-1)

 QUICKSORT(A, q+1, r)

}

PARTITION(A, p, r)

{

 x = A[r]

 i = p-1

 for j = p to r-1

 if A[j] ≤ x

 i = i+1

 exchange A[i] with A[j]

 exchange A[i+1] with A[r]

 return i+1

}

Bright

Date _____

BINARY SEARCH

```
BinarySearch(int arr[], int l, int r, int x)
```

```
{
```

```
    if (r >= 1) {
```

```
        int mid = (l + (r - 1)) / 2;
```

```
        if (arr[mid] == x)
```

```
            return mid;
```

```
        if (arr[mid] > x)
```

```
            return BinarySearch(arr, l, mid - 1, x);
```

```
        return BinarySearch(arr, mid + 1, r, x);
```

```
    }
```

```
    return -1;
```

```
}
```

Bright

Date _____

HEAP SORT

MaxHeapify(A, i, n)

{

$l \leftarrow \text{LEFT}(i)$

$r \leftarrow \text{RIGHT}(i)$

if ($l \leq n$ && $A[l] > A[i]$)

$\text{largest} \leftarrow l$

else $\text{largest} \leftarrow i$

if ($r \leq n$ && $A[r] > A[\text{largest}]$)

$\text{largest} \leftarrow r$

if ($\text{largest} \neq i$)

exchange $A[i] \leftrightarrow A[\text{largest}]$

MAX-Heapify($A, \text{largest}, n$)

}

BuildMaxHeap(A)

{

$n = \text{length}[A]$

for ($i \leftarrow [n/2]$ to 1)

do MaxHeapify(A, i, n)

}

Bright

Date _____

COUNT SORT

CountSort(A, B, K)

{

let $C[1 \dots K]$ be a new array

for ($i = 1$ to K)

$C[i] = 0$

for ($j = 1$ to $A.length$)

$C[A[j]] = C[A[j]] + 1$

for ($i = 2$ to K)

$C[i] = C[i] + C[i-1]$

for ($j = A.length$ to 1)

$B[C[A[j]]] = A[j]$

$C[A[j]] = C[A[j]] - 1$

}

Bright

Date_____

RADIX SORT (Similar to Count Sort)

Date _____

BFS

BFS(s);

{ visit(s);

queue.insert(s);

while (queue is not empty)

{

u = queue.extractHead();

for (each edge $\langle u, d \rangle$)

{

if (d has not been visited)

{ visit(d);

queue.insert(d);

}

}

}

}

Bright

Date _____

DFS

DFS(s)

{

s.underDFS = true;

for (each edge $\langle s, d \rangle$)

{

if (!d.underDFS and d have not been visited)

{

DFS(d);

}

}

visit(s);

}

Bright

Date _____

PRIM'S ALGORITHM

PRIM (Graph (V, E), S)

{

MST = {}

visited = {S}

for all v besides S : $d[v] = \infty$ and $K[v] = \text{NULL}$

for each neighbor v of S : $d[v] = w(S, v)$ and $K[v] = S$

while $\text{len}(\text{visited}) < n$:

x = unvisited vertex v with smallest $d[v]$ value

MST.add($(K[x], x)$)

for each unvisited neighbor v of x:

$d[v] = \min(w(x, v), d[v])$

if $d[v]$ was updated : $K[v] = x$

visited.add(x)

return MST;

}

Date _____

KRUSKAL'S ALGORITHM

Kruskal (Graph(V, E))

{

E -Sorted = E sorted by weight in non-decreasing order

$MST = \{\}$

for v in V :

Make-SET(v)

for (u, v) in E -SORTED:

if $FIND(u) \neq FIND(v)$

$MST.add(u, v)$

UNION(u, v)

return MST

}

Bright

Date _____

BELLMAN-FORD ALGORITHM

BellmanFord(G, s)

{

$d^{(k)} = []$ for $k=0, \dots, n-1$

$d^{(0)}[v] = \infty$ for all v in V (except s)

$d^{(0)}[s] = 0$

for $k=1, \dots, n-1$;

for b in V :

$d^{(k)}[b] \leftarrow \min \{ d^{(k-1)}[b], \min_a \{ d^{(k-1)}[a] + w(a, b) \} \}$

return $d^{(n-1)}$

}

Date _____

DIJKSTRA'S ALGORITHM

Dijkstra(G, s)

{

dist[s] $\leftarrow$ 0

for all $v \in V \setminus \{s\}$

do dist[v] $\leftarrow \infty$

$S \leftarrow \emptyset$

$Q \leftarrow V$

while $Q \neq \emptyset$

do $u \leftarrow \text{mindistance}(Q, \text{dist})$

$S \leftarrow S \cup \{u\}$

for all $v \in \text{neighbor}[u]$ do

dist[v] = $\min(\text{dist}[v], \text{dist}[u] + w(u, v))$

return dist;

}

Bright

Date _____

FLOYD - WARSHALL ALGORITHM

Floyd ($W[1 \dots n, 1 \dots n]$)

{

$D \leftarrow W$

for $k \leftarrow 1$ to n do

for $i \leftarrow 1$ to n do

for $j \leftarrow 1$ to n do

$D[i, j] \leftarrow \min\{D[i, j], D[i, k] + D[k, j]\}$

return D ;

}

- **APPROX-VERTEX-COVER(G)**

$C = \emptyset;$

$E' = G.E;$

while($E' \neq \emptyset$) {

 Randomly choose a edge (u,v) in E' , put u and v into C ;

 Remove all the edges that covered by u or v from E'

}

Return C ;

Traveling-salesman problem

APPROX-TSP-TOUR(G)

Minimum Spanning Tree;

Creating a Cycle;

Removing Redundant Visits;

The set-covering problem

GREEDY-SET-COVER(X, F)

$U = X;$

$C = \emptyset;$

While($U \neq \emptyset$) {

 Select $S \in F$ that maximizes $|S \cap U|;$

$U = U - S;$

$C = C \cup \{S\};$

}

return $C;$

GRAHAM-SCAN(Q)

- 1 let p_0 be the point in Q with the minimum y -coordinate,
or the leftmost such point in case of a tie
- 2 let $\langle p_1, p_2, \dots, p_m \rangle$ be the remaining points in Q ,
sorted by polar angle in counterclockwise order around p_0
(if more than one point has the same angle, remove all but
the one that is farthest from p_0)
- 3 let S be an empty stack
- 4 PUSH(p_0, S)
- 5 PUSH(p_1, S)
- 6 PUSH(p_2, S)
- 7 for $i = 3$ to m
- 8 while the angle formed by points NEXT-TO-TOP(S), TOP(S),
 and p_i makes a nonleft turn
- 9 POP(S)
- 10 PUSH(p_i, S)
- 11 return S

Closest Pair Algorithm

Closest-Pair($p_1, \dots, p_n$)

{

 Compute separation line L such that half the points are on one side and half on the other side.

$O(N \log N)$

$\delta_1 = \text{Closest-Pair}(\text{left half})$

$\delta_2 = \text{Closest-Pair}(\text{right half})$

$2T(N/2)$

$\delta = \min(\delta_1, \delta_2)$

 Delete all points further than δ from separation line L

$O(N)$

 Sort remaining points by y-coordinate.

$O(N \log N)$

 Scan points in y-order and compare distance between each point and next 11 neighbors. If any of these distances is less than δ , update δ .

$O(N)$

 return δ .

}



Brute-Force Algorithm



<http://whocouldthat.be/visualizing-string-matching/>

```
function brute_force(text[], pattern[])
{
    // let n be the size of the text and m the size of the
    // pattern

    for(i = 0; i < n; i++) {
        for(j = 0; j < m && i + j < n; j++)
            if(text[i + j] != pattern[j]) break;
        // mismatch found, break the inner loop
        if(j == m) // match found
    }
}
```

Algorithm *KMPMatch*(*T*, *P*)

$F \leftarrow \text{failureFunction}(P)$

$i \leftarrow 0$

$j \leftarrow 0$

while $i < n$

if $T[i] = P[j]$

if $j = m - 1$

return $i - j$ { match }

else

$i \leftarrow i + 1$

$j \leftarrow j + 1$

else

if $j > 0$

$j \leftarrow F[j - 1]$

else

$i \leftarrow i + 1$

return -1 { no match }

Failure



The Boyer-Moore

Algorithm *BoyerMooreMatch*(T, P, Σ)

$L \leftarrow \text{lastOccurrenceFunction}(P, \Sigma)$

$i \leftarrow m - 1$

$j \leftarrow m - 1$

repeat

 if $T[i] = P[j]$

 if $j = 0$

 return i { match at i }

 else

$i \leftarrow i - 1$

$j \leftarrow j - 1$

 else

 { character-jump }

$l \leftarrow L[T[i]]$

$i \leftarrow i + m - \min(j, 1 + l)$

$j \leftarrow m - 1$

until $i > n - 1$

return -1 { no match }

Pattern Ma

